UniSENAI

Trilha de Banco de Dados

Módulo 10: Triggers e Eventos

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.

2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.

3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.

4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.

5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.

6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.

7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.

8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 10: Triggers e Eventos

Os **triggers** e **eventos** no MySQL são mecanismos de **automação interna do banco de dados**, permitindo executar ações automaticamente com base em **mudanças nos dados** ou em **horários programados**. Esses recursos são amplamente utilizados para manter a integridade dos dados, automatizar rotinas e reduzir dependências externas.

Este módulo aborda:

- Criação e uso de **triggers**
- Automação com **eventos agendados**
- Boas práticas e cuidados em ambientes reais

---

1. Triggers

O que são Triggers

Um **trigger** é um tipo especial de procedimento armazenado que é executado **automaticamente** quando ocorre um evento específico em uma tabela, como:

- `INSERT`
- `UPDATE`
- `DELETE`

Triggers são associados diretamente a uma tabela e não podem ser executados manualmente.

Vantagens dos Triggers

- Automatizam processos internos
- Mantêm a integridade dos dados
- Garantem execução de regras automaticamente
- Reduzem dependência da aplicação

1.2 Criação de Triggers

Sintaxe para Criação

```
CREATE TRIGGER nome_trigger
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON nome_da_tabela
FOR EACH ROW
corpo_do_trigger;
```

- **`BEFORE`**: O trigger é executado antes da operação.

- **AFTER**: O trigger é executado após a operação.
- **INSERT**, **UPDATE**, **DELETE**: Define o evento que dispara o trigger.

Exemplo Prático

Criar um trigger que insere um registro de log toda vez que um pedido for inserido:

```
CREATE TRIGGER log_pedido_inserido
AFTER INSERT ON pedidos
FOR EACH ROW
BEGIN
    INSERT INTO log_acoes (acao, data_hora)
    VALUES ('Pedido inserido', NOW());
END;
```

- **Tabela `log_acoes`**:
  - `acao` (texto da ação realizada).
  - `data_hora` (momento em que a ação ocorreu).

1.3 Exclusão de Triggers

Para excluir um trigger, use o comando `DROP TRIGGER`.

Exemplo: Excluir o trigger `log_pedido_inserido`:

```
DROP TRIGGER log_pedido_inserido;
```

1.4 Tipos de Triggers

- **BEFORE**:
  - Executado antes da operação.
  - **Exemplo**: Validar dados antes de inserir.

```
CREATE TRIGGER valida_estoque
BEFORE INSERT ON pedidos
FOR EACH ROW
BEGIN
    IF NEW.quantidade <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A quantidade deve ser maior que zero.';
    END IF;
END;
```

2. AFTER:

- Executado após a operação.
- **Exemplo**: Atualizar o estoque após uma venda.

```
CREATE TRIGGER atualiza_estoque
AFTER INSERT ON vendas
FOR EACH ROW
BEGIN
    UPDATE produtos
    SET estoque = estoque - NEW.quantidade
    WHERE id_produto = NEW.id_produto;
END;
```

---

2. Eventos (Event Scheduler)

2.1 O que são Eventos no MySQL

Um **evento** é uma tarefa agendada que é executada automaticamente em um **horário específico** ou em **intervalos regulares**, funcionando como um **cron interno do banco de dados**.

Vantagens dos Eventos

- Automatizam tarefas recorrentes
- Reduzem dependência de scripts externos
- Ideal para manutenção e rotinas periódicas

2.2 Criação de Eventos Programados

Sintaxe Geral

```
CREATE EVENT nome_evento
ON SCHEDULE {AT data_hora | EVERY intervalo}
DO
comando_sql;
```

- **ON SCHEDULE**:
  - **AT**: Especifica uma data e hora exata.
  - **EVERY**: Define um intervalo regular.
- **DO**: Contém o comando SQL que será executado.

Exemplo Prático

1. **Limpar Registros Antigos de Log**
   Criar um evento que apaga registros de log com mais de 30 dias, executado diariamente:

```
CREATE EVENT limpa_logs
ON SCHEDULE EVERY 1 DAY
DO
DELETE FROM log_acoes WHERE data_hora < NOW() - INTERVAL 30 DAY;
```

2. **Enviar Relatórios Automáticos** Criar um evento para gerar um relatório de vendas todo primeiro dia do mês:

```
CREATE EVENT gera_relatorio_vendas
ON SCHEDULE EVERY 1 MONTH
STARTS '2024-01-01 00:00:00'
DO
INSERT INTO relatorios (descricao, data_geracao)
VALUES ('Relatório mensal de vendas', NOW());
```

2.3 Ativação do Agendador de Eventos

Por padrão, o agendador de eventos pode estar desativado. Ative-o com o comando:

```
SET GLOBAL event_scheduler = ON;
```

Verifique o status do agendador:

```
SHOW VARIABLES LIKE 'event_scheduler';
```

---

3. Aplicabilidade Prática

- **Triggers**:
  - Automatizar atualizações em tabelas relacionadas.
  - Monitorar alterações e gerar logs.
  - Implementar validações automáticas.

Exemplo: Gerar logs automáticos ao excluir usuários.

```
CREATE TRIGGER log_usuario_excluido
AFTER DELETE ON usuarios
FOR EACH ROW
BEGIN
    INSERT INTO log_acoes (acao, data_hora)
    VALUES (CONCAT('Usuário excluído: ', OLD.nome), NOW());
END;
```

- **Eventos:**
  - Manutenção automatizada do banco de dados.
  - Limpeza periódica de dados desnecessários.
  - Geração de relatórios automáticos.

Exemplo: Um sistema de vendas que limpa carrinhos de compras abandonados após 24 horas.

```
CREATE EVENT limpa_carrinhos
ON SCHEDULE EVERY 1 DAY
DO
DELETE FROM carrinhos WHERE data_criacao < NOW() - INTERVAL 1 DAY;
```

---

4. Melhores Práticas

Checklist Técnico

✔ Manter triggers simples e bem documentados
✔ Evitar lógica complexa em triggers
✔ Testar triggers e eventos em ambiente de homologação

✔ Monitorar impacto em performance
✔ Documentar claramente a finalidade
✔ Usar eventos apenas para tarefas realmente periódicas

---

5. Quando NÃO Usar Triggers ou Eventos

❌ Regras complexas de negócio difíceis de manter
❌ Lógicas que dependem de múltiplas tabelas e fluxos
❌ Processos que exigem controle detalhado de erro
❌ Quando a lógica deve ficar exclusivamente na aplicação

Questões Práticas

1. **Criação de um Trigger BEFORE**
   Crie um trigger que valide, antes de inserir um pedido na tabela `pedidos`, se a quantidade solicitada de um produto é menor ou igual ao estoque disponível.
2. **Criação de um Trigger AFTER**
   Desenvolva um trigger que atualize o estoque na tabela `produtos` sempre que um novo pedido for registrado na tabela `pedido_produto`.
3. **Registro de Logs**
   Crie um trigger que registre em uma tabela `log_acoes` toda vez que um registro for excluído da tabela `clientes`. O log deve incluir a data/hora e a identificação do cliente excluído.
4. **Exclusão de um Trigger**
   Exclua o trigger que registra exclusões de clientes na tabela `log_acoes`.
5. **Limpeza Automática com Eventos**
   Crie um evento que limpe registros da tabela `log_acoes` que tenham mais de 60 dias, executado diariamente às 23h59.
6. **Geração Automática de Relatórios**
   Configure um evento que gere relatórios mensais de vendas, inserindo informações na tabela `relatorios` todo dia 1º de cada mês, à meia-noite.
7. **Ativação do Agendador de Eventos**
   Ative o agendador de eventos no MySQL e verifique se ele está funcionando corretamente.
8. **Modificação de um Trigger Existente**
   Altere um trigger existente para incluir a validação de um novo campo antes de realizar a inserção.
9. **Teste de Performance de um Trigger**
   Execute um teste de performance em um trigger que é acionado frequentemente. Proponha melhorias para otimizar sua execução.
10. **Documentação de um Evento**
    Documente detalhadamente um evento que gere relatórios automáticos, explicando sua finalidade, comportamento esperado e benefícios para o sistema.

Questões Teóricas

- **Conceito de Triggers**
  Explique o que são triggers no MySQL e cite pelo menos duas vantagens do seu uso.
- **Tipos de Triggers**
  Qual a diferença entre triggers `BEFORE` e `AFTER`? Cite exemplos de situações em que cada um seria utilizado.
- **Vantagens dos Eventos**
  Liste três vantagens de usar eventos no MySQL em vez de cron jobs externos.
- **Validação com Triggers**
  Como um trigger pode ser usado para validar dados antes de uma operação ser realizada? Explique com um exemplo teórico.
- **Impacto de Triggers na Performance**
  Quais são os cuidados a serem tomados ao criar triggers para evitar impactos negativos na performance do banco de dados?
- **Automatização de Tarefas com Eventos**
  Explique como eventos podem ser usados para automatizar a manutenção do banco de dados. Dê um exemplo prático.
- **Ativação do Agendador de Eventos**
  Por que o agendador de eventos pode estar desativado por padrão no MySQL? Como ativá-lo?
- **Exclusão de Triggers**
  Qual é a sintaxe para excluir um trigger no MySQL? O que acontece com os dados afetados por triggers excluídos?
- **Melhores Práticas para Triggers**
  Quais são as melhores práticas para a criação de triggers? Explique como elas podem beneficiar a manutenção do banco de dados.
- **Planejamento de Eventos**
  Por que é importante planejar eventos no banco de dados? Cite situações em que um evento mal configurado pode gerar problemas.